



The Coordination of Scheduling and Batch Deliveries

NICHOLAS G. HALL

hall.33@cob.osu.edu

*Fisher College of Business, The Ohio State University, 2100 Neil Avenue, Columbus, Ohio,
OH 43210-1144, USA*

CHRIS N. POTTS

C.N.Potts@maths.soton.ac.uk

School of Mathematics, University of Southampton, Southampton, SO 17 1BJ, UK

Abstract. This paper considers several scheduling problems where deliveries are made in batches with each batch delivered to the customer in a single shipment. Various scheduling costs, which are based on the delivery times of the jobs, are considered. The objective is to minimize the scheduling cost plus the delivery cost, and both single and parallel machine environments are considered. For many combinations of these, we either provide efficient algorithms that minimize total cost or show that the problem is intractable. Our work has implications for the coordination of scheduling with batch delivery decisions to improve customer service.

Keywords: manufacturing and scheduling, batching and delivery, sequencing, dynamic programming

Introduction

In production scheduling, *batching* is the decision about whether or not to schedule certain jobs consecutively. A first motivation for batching is to reduce setup times or costs that occur when dissimilar items are produced consecutively. Many well documented examples arise in the chemical, food processing and fertilizer industries. Of particular note is the manufacture of engineering parts, where the principles of group technology (Ham, Hitomi, and Yoshida, 1985) are used to divide parts into families such that machine setups are not needed between the production of parts in the same family. Second, in multistage production systems such as flow shops, the batching of items for transportation between machines may improve machine utilization where the transportation device is a bottleneck. Third, there are applications where several jobs are naturally processed simultaneously in a batch, for example the “burn-in” testing operation in semiconductor manufacturing (Lee, Uzsoy, and Martin-Vega, 1992). A fourth motivation arises in applications where an order consists of many similar items for the same customer. Here, batching only part of the order for delivery may improve customer service, compared to waiting until the entire order has been processed.

This paper follows the last motivation in the above paragraph. We make the natural assumption that the scheduling cost of a job is determined by the time at which the batch containing it is delivered to the customer. We also generalize existing models by

considering the cost of batch deliveries. This objective provides a natural extension of the classical scheduling literature to allow for batch delivery cost.

Potts and Van Wassenhove (1992) provide a review of the literature which integrates scheduling with batching and lot sizing. They observe that scheduling models usually assume that batching and lot sizing decisions have already been made, whereas batching and lot sizing models rarely consider sequencing issues. As part of the motivation for considering integrated scheduling and batching models, they mention computer integrated manufacturing environments, where scheduling and batching decisions need to be controlled by the same computer.

Webster and Baker (1995) review the literature on scheduling and batching models. They distinguish between two alternative assumptions that may apply for this class of problems. The first is *batch availability*, under which a job only becomes available for later processing or dispatch to the customer when the entire batch of which it is part has been processed. Under the alternative *job availability* assumption, however, a job becomes available once it has been processed. This paper uses the batch availability assumption.

Potts and Kovalyov (2000) review the much more extensive literature on scheduling and batching that is now available. They provide a classification of problems that are solvable in polynomial or pseudopolynomial time, or are binary or unary *NP*-hard (see Garey and Johnson, 1979, for related definitions). They focus particularly on the usefulness of dynamic programming as a method for solving a variety of scheduling and batching problems. Our paper makes extensive use of this general technique. Finally, the authors suggest the need to study problems which combine scheduling and batching cost, as in our paper.

We briefly discuss some works related to integrated scheduling and batching decisions. Bruno and Downey (1978) provide a variety of complexity results for sequencing problems with setup times and changeover costs. Monma and Potts (1989) derive results about the sequencing of jobs within a family, for single machine problems with various objectives and sequence-dependent setup times and costs, and propose several dynamic programming algorithms. Albers and Brucker (1993) derive several complexity results for sequencing jobs on a single machine under batch availability to minimize total weighted completion time, where a setup time is required for each batch before it can start processing. Herrmann and Lee (1993) study a single machine scheduling problem with a common due date, where tardy jobs are delivered in batches with a fixed cost. They describe a dynamic programming algorithm and identify several efficiently solvable special cases. Cheng, Gordon, and Kovalyov (1996) consider a single machine batch scheduling problem, in which they minimize the sum of batch delivery cost plus job earliness penalties. A similar model but with given batch delivery dates is analyzed by Yang (2000). Hall, Lesaoana, and Potts (2001) also consider an environment in which delivery dates are fixed in advance of detailed schedule planning. This environment is closely related to batching because, at each fixed delivery date, all the completed jobs are delivered in a batch to the customer. For a wide variety of objectives, the authors provide either a polynomial time algorithm or a proof of intractability. Lee and Chen

(2001) study the coordination of scheduling and transportation decisions, taking into account both transportation capacities and times. They provide a variety of algorithms and intractability results. In the context of supply chains, Hall and Potts (2003) analyze the complexity of some single machine scheduling problems with batch deliveries, but without a transporter availability constraint.

This paper is organized as follows. In Section 1, we describe our notation and our scheme for the classification of scheduling and batch delivery problems. We also provide some preliminary results that simplify the later analysis, and present an overview of our results. In Section 2, we provide algorithms and computational complexity results for the minimization of total scheduling and delivery cost, using a variety of different scheduling objectives, in a single machine manufacturing environment. In Section 3, we provide similar results for identical parallel machine problems. Section 4 contains a conclusion and some suggestions for future research.

1. Preliminaries

In Section 1.1, we describe our notation and assumptions. Section 1.2 provides some general results that simplify the subsequent analysis. Section 1.3 presents an overview of the results in the paper.

1.1. Notation and classification

We begin with some notation. The jobs are processed either by a single machine or by two identical parallel machines. In the latter case, we denote the two machines by M_1 and M_2 . Let $N = \{1, \dots, n\}$ denote the set of jobs to be processed. For job $j \in N$, let p_j denote its processing time and w_j its weight (or value), where we let $P = \sum_{j=1}^n p_j$ and $W = \sum_{j=1}^n w_j$. In some of the problems, job j also has a due date d_j . We assume throughout that p_j , w_j and d_j are integers, for $j \in N$. Preemption is not allowed. A group of jobs forms a *batch* if all of these jobs are dispatched to a customer at the same time in a single delivery. Let D denote the nonnegative cost of delivering a batch. Also, let T denote the minimum time between any two consecutive deliveries, where we assume that $T \leq P$. The value of T is defined by the amount of time that it takes to load a transporter if there is a fleet of transporters, or by the travel time to and from the customers plus loading time if there is only a single transporter.

For any schedule σ , we also define:

$C_j(\sigma)$ = the time at which job j is delivered to its customer;

$L_j(\sigma) = C_j(\sigma) - d_j$, the lateness of job j ;

$U_j(\sigma) = \begin{cases} 0 & \text{if job } j \text{ is delivered to its customer by its due date,} \\ 1 & \text{if job } j \text{ is late;} \end{cases}$

$T_j(\sigma) = \max\{C_j(\sigma) - d_j, 0\}$, the tardiness of job j ;

$y(\sigma)$ = the number of deliveries in σ .

When there is no ambiguity, we simplify $C_j(\sigma)$, $L_j(\sigma)$, $U_j(\sigma)$, $T_j(\sigma)$ and $y(\sigma)$ to C_j , L_j , U_j , T_j and y , respectively. We also refer to C_j as the *completion time* of job j (as perceived by the customer). Some of our algorithms use the *makespan* of a (partial) schedule to define when deliveries occur, where the makespan is the time when the last job in the partial schedule completes processing. In some of our analysis, we use a shortest processing time (SPT) ordering of the jobs, where they are scheduled in nondecreasing order of their processing times. We also use an earliest due date (EDD) ordering, where the jobs are scheduled in nondecreasing order of their due dates.

The standard classification scheme for scheduling problems (Graham et al., 1979) is $\alpha|\beta|\gamma$, where α indicates the scheduling environment, β describes the job characteristics or restrictive requirements, and γ defines the objective function to be minimized. Under α , we may have either ‘1’ which denotes a single machine, or ‘P2’ which denotes an environment with two identical parallel machines. Under β , we may have ‘T’, which denotes that there is a constraint on the minimum time interval between consecutive deliveries; otherwise, $T = 0$ is assumed.

The objective functions that we consider under γ comprise a delivery cost Dy and a scheduling cost as follows:

- $\sum(w_j)C_j$ = the total (weighted) completion time of the jobs;
- $L_{\max} = \max_{j \in N}\{L_j\}$, the maximum lateness of the jobs;
- $\sum(w_j)U_j$ = the total (weighted) number of late jobs;
- $\sum T_j$ = the total tardiness of the jobs.

An example of the classification scheme is problem $1|T|Dy + \sum C_j$, which denotes the minimization of the total of delivery cost and job completion times on a single machine, under the transporter availability constraint.

1.2. General results

We make use of the following result which is stated without proof.

Lemma 1. In any of the scheduling problems that we consider, there exists an optimal schedule without idle time between jobs on any machine.

The following result describes a candidate set of possible delivery times.

Lemma 2. In any of the scheduling problems that we consider, there exists an optimal schedule in which all deliveries are made either at job completion times or immediately the transporter becomes available.

Proof. Assume that there is a delivery which is scheduled neither at a job completion time nor at a time when the transporter becomes available. This delivery can be moved to the latest earlier time which fits either of those conditions. Since the same jobs can be

delivered at that earlier time, and there are no additional deliveries, the total cost is not increased. $\square$

Lemma 3. In any of the scheduling problems with the transporter availability constraint that we consider, there exists an optimal schedule in which the last delivery is started before time $M + T$ and the transporter completes the last delivery before time $M + 2T$, where M is the time when the last job completes processing, and $M \leq P$.

Proof. The latest delivery that does not include the last job to complete processing occurs before time M , and from Lemma 1 we have $M \leq P$. Therefore, the transporter becomes available before time $M + T$ to perform the final delivery. $\square$

1.3. Overview of the results

Because the cost and the feasibility of a schedule can be evaluated in $O(n)$ time, the recognition versions of all the problems considered in this paper belong to the class NP . Table 1 presents a summary of our algorithm and computational complexity results for scheduling and batch delivery problems. The first column describes the problem type. The second column shows the running time of the fastest known polynomial or pseudopolynomial time algorithm for the problem, if such an algorithm exists. Otherwise, we use *BNPC* or *UNPC* to indicate that the equivalent recognition version of a problem is NP -complete with respect to a binary or unary encoding of the data, respectively. Related definitions can be found in Garey and Johnson (1979). The third column contains a reference to where a proof of that result can be found. We refer to Hall and Potts (2003) for related complexity results for problems with no transporter availability constraint.

Table 1
The complexity of scheduling and batch delivery problems.

Problem	Complexity	Reference
$1 T Dy + \sum C_j$	$O(n^4)$	Thm 1
$1 Dy + \sum w_j C_j$	<i>UNPC</i>	Thm 2
$1 T Dy + \sum w_j C_j$	<i>UNPC</i>	Thm 2
$1 T Dy + L_{\max}$	$O(n^5)$	Thm 3
$1 T Dy + \sum U_j$	$O(n^8 \log n)$	Thm 4
$1 T Dy + \sum w_j U_j$	<i>BNPC</i> , $O(n^2 P \min\{P, nW\})$	Karp (1972), Thm 5
$1 Dy + \sum T_j$	<i>BNPC</i> , $O(n^4 P^4)$	Du and Leung (1990), Thm 6
$1 T Dy + \sum T_j$	<i>BNPC</i> , Open	Du and Leung (1990)
$P2 Dy + \sum C_j$	<i>BNPC</i> , Open	Thm 7
$P2 T Dy + \sum C_j$	<i>BNPC</i> , Open	Thm 7
$P2 Dy + L_{\max}$	<i>BNPC</i> , $O(n^3 P)$	Lenstra et al. (1977), Thm 8
$P2 T Dy + L_{\max}$	<i>BNPC</i> , $O(n^3 P^2)$	Lenstra et al. (1977), Thm 9
$P2 Dy + \sum w_j U_j$	<i>BNPC</i> , $O(n^2 P \min\{P, nW\})$	Lenstra et al. (1977), Thm 10
$P2 T Dy + \sum w_j U_j$	<i>BNPC</i> , $O(n^2 P^3 \min\{P, nW\})$	Lenstra et al. (1977), Thm 11

2. Single machine problems

In this section, we describe models for scheduling a single machine to minimize the sum of scheduling and delivery cost in various environments. For many of the models, we establish the processing order of the jobs in an optimal schedule, and then use dynamic programming to form batches for delivery. The dynamic programs are forward algorithms that append either a job or a batch to a previous partial schedule. In many cases, the dynamic program is specified by a recursion. However, sometimes it is more convenient to present the algorithm in an alternative form where each state is used in turn to generate a new state. The new state, together with its associated value, is a *trial state*. A trial state is replaced by another trial state with the same values of the state variables and a lower cost.

The names of our algorithms in this section begin with a letter that indicates the scheduling cost that is being considered. The second letter is “U” (unconstrained) or “C” (constrained), depending on whether the algorithm applies to the version of the problem without or with (respectively) a constraint on the times of consecutive deliveries.

2.1. Sum of completion times

Our first result establishes that we can focus on processing the jobs in shortest processing time (SPT) order.

Lemma 4. For problem $1|T|Dy + \sum C_j$, cost is minimized by sequencing the jobs according to a shortest processing time (SPT) rule.

Proof. Consider an optimal schedule σ^* . At each delivery, all jobs that are processed but not delivered are included in the delivery batch. Since all jobs within a delivery batch can be sequenced in SPT order without affecting the cost, we assume that this SPT property for jobs within a batch holds for σ^* . If σ^* contains jobs that are not sequenced in SPT order, then we must have a job j that is the last job to be processed in some batch B , and another job i that is the first job to be processed in the next batch B' , where $p_j > p_i$. Consider a schedule σ that is created by interchanging jobs j and i , and forming delivery batches containing jobs $B \cup \{i\} \setminus \{j\}$ and $B' \cup \{j\} \setminus \{i\}$ in σ , where the first of these batches is delivered at the same time as batch B in σ^* and the second of these batches is delivered at the same time as batch B' in σ^* . All other delivery batches are identical and delivered at the same time in σ as in σ^* . Thus, the cost associated with σ is equal to the cost associated with σ^* . A finite number of repetitions of this argument establishes that there exists an optimal schedule in which the jobs are sequenced in SPT order. $\square$

As a result of Lemma 4, we assume when considering problem $1|T|Dy + \sum C_j$ that the jobs are indexed in SPT order, i.e. such that $p_1 \leq \dots \leq p_n$. We compute $P_j = \sum_{i=1}^j p_i$, the time at which job j completes processing, for $j = 1, \dots, n$, and define $P_0 = 0$. This preprocessing step requires $O(n \log n)$ time, which does not affect the overall time complexity of our algorithm.

Hall and Potts (2003) describe an $O(n^2)$ dynamic programming algorithm for problem $1||Dy + \sum C_j$. However, in the constrained version of this problem, $1|T|Dy + \sum C_j$, we require additional state variables to indicate when the transporter next becomes available. To maintain a polynomial time complexity, we introduce a state variable h which specifies the last job that is delivered immediately after its processing completes, and a state variable v that specifies the number of subsequent deliveries. Thus, the transporter performs its last delivery in the (partial) schedule at time $P_h + vT$.

Algorithm CC

Value Function

$f(j, h, v)$ = the minimum total cost for processing and delivering jobs $1, \dots, j$, where the last delivery when a job completes processing occurs at time P_h and there are v subsequent deliveries, so that the batch containing job j is delivered at time $P_h + vT$, where $v \leq j - h$, $h \leq j$ and $h = j$ if $v = 0$.

Boundary Condition

$$f(j, j, 0) = \begin{cases} 0, & \text{for } j = 0, \\ D + jP_j, & \text{for } j > 0 \text{ and } P_j < T. \end{cases}$$

Optimal Solution Value

$$\min_{1 \leq h \leq n, 0 \leq v \leq n} \{f(n, h, v)\}.$$

Recurrence Relations

$$\begin{aligned} f(j, j, 0) &= \min_{0 \leq h \leq i < j, 1 \leq v \leq \min\{i-h+1, V\}} \{(j-i)P_j + D + f(i, h, v-1)\}, \quad \text{for } P_j \geq T, \\ f(j, h, v) &= \min_{0 \leq i < j} \{(j-i)(P_h + vT) + D + f(i, h, v-1)\}, \quad \text{for } h < j \text{ and } v > V, \\ &\text{where } V = \lfloor (P_j - P_h)/T \rfloor. \end{aligned}$$

The boundary condition considers all possible first batches that are delivered before time T . In the first recurrence relation, a delivery occurs at time P_h and then at each T time units until time $P_h + (v-1)T$, and then another delivery for batch $\{i+1, \dots, j\}$ occurs at time $P_j \geq P_h + vT$. The second recurrence relation represents the case where deliveries are previously scheduled at times $P_h, P_h + T, \dots, P_h + (v-1)T$, and then batch $\{i+1, \dots, j\}$ is delivered at time $P_h + vT > P_j$.

Theorem 1. Algorithm CC finds an optimal schedule for problem $1|T|Dy + \sum C_j$ in $O(n^4)$ time.

Proof. The proof of optimality follows from Lemmas 1, 2 and 4. By definition, $j, h, v \leq n$. Consequently, the first recurrence relation with $h = j$ and $v = 0$ requires $O(n^3)$ time

for each j . The second recurrence relation requires $O(n)$ time for each j, h and v . Therefore, the overall time complexity of Algorithm CC is $O(n^4)$. $\square$

We now show that the corresponding problem with total weighted completion time as the scheduling objective is intractable, even if there is no transporter availability constraint.

Theorem 2. The recognition versions of problems $1||Dy + \sum w_j C_j$ and $1|T|Dy + \sum w_j C_j$ are unary NP -complete.

Proof. Since the second problem is a generalization of the first, we consider problem $1||Dy + \sum w_j C_j$. We use a reduction from the following problem which is known to be unary NP -complete.

3-Partition (Garey and Johnson, 1979): given $3m$ elements with integer sizes $a_1, \dots, a_{3m}$, where $\sum_{i=1}^{3m} a_i = mz$ and $z/4 < a_i < z/2$ for $i = 1, \dots, 3m$, does there exist a partition $S_1, \dots, S_m$ of the index set $\{1, \dots, 3m\}$ such that $|S_j| = 3$ and $\sum_{i \in S_j} a_i = z$ for $j = 1, \dots, m$?

We assume without loss of generality that, if there exists a solution to 3-Partition, then the elements are numbered such that $a_{3i-2} + a_{3i-1} + a_{3i} = z$ for $i = 1, \dots, m$.

Consider an instance of the decision version of problem $1||Dy + \sum w_j C_j$ defined by:

$$\begin{aligned} n &= 3m, & p_i &= w_i = a_i \quad \text{for } i = 1, \dots, n, \\ D &= z^2/2, & \text{and } C &= m(m+2)z^2/2, \end{aligned}$$

where C is a threshold cost. We prove that there exists a schedule for this instance of $1||Dy + \sum w_j C_j$ with cost less than or equal to C if and only if there exists a solution to 3-Partition.

($\Rightarrow$) Consider the sequence $(1, \dots, n)$ with deliveries when job 3_j , completes processing for $j = 1, \dots, m$. This yields $Dy + \sum w_j C_j = z \cdot z + z \cdot 2z + \dots + z \cdot mz + mz^2/2 = C$. ($\Leftarrow$) We first show that a cost minimizing solution with h batches has equal total processing time in each batch. We can formulate this problem as:

$$\begin{aligned} \text{minimize} \quad & x_1^2 + (x_1 + x_2)x_2 + \dots + (x_1 + \dots + x_h)x_h + hz^2/2 \\ \text{subject to} \quad & x_1 + \dots + x_h = mz, \end{aligned}$$

where x_i is the total processing time in batch i , for $i = 1, \dots, h$. The objective function can be expressed as

$$\left(\sum_{i=1}^h x_i \right)^2 / 2 + \sum_{i=1}^h x_i^2 / 2 + hz^2/2 = m^2 z^2 / 2 + \sum_{i=1}^h x_i^2 / 2 + hz^2/2.$$

This second term is minimized by setting $x_1 = \dots = x_h = mz/h$. Therefore, the total cost for h deliveries is greater than or equal to

$$m^2 z^2 / 2 + m^2 z^2 / (2h) + hz^2/2.$$

This expression achieves a minimum with respect to h at $h = m$, and this minimum value is equal to C . Moreover, the minimum total cost of C is achieved only for $h = m$ and identical processing times for each of the m batches. From the definition of 3-Partition, each batch must contain exactly three jobs. This implies the existence of a solution to 3-Partition. $\square$

2.2. Maximum lateness

In this subsection, we adapt Algorithm CC of Section 2.1 to the case where the scheduling objective is the maximum lateness.

Lemma 5. For problem $1|T|Dy + L_{\max}$, there exists an optimal schedule in which the jobs are sequenced according to an earliest due date (EDD) rule.

Proof. Consider an optimal schedule σ^* . At each delivery, all jobs that are processed but not delivered are included in the delivery batch. Since all jobs within a delivery batch can be sequenced in EDD order without affecting the cost, we assume that this EDD property for jobs within a batch holds for σ^* . If σ^* contains jobs that are not sequenced in EDD order, then we must have a job j that is the last job to be processed in some batch B , and another job i that is the first job to be processed in the next batch B' , where $d_j > d_i$. Consider a schedule σ in which delivery batches $B \setminus \{j\}$ and $B' \cup \{j\}$ are formed, where the first of these batches is delivered at the same time as batch B in σ^* (assuming that the batch is non-empty) and the second of these batches is delivered at the same time as batch B' in σ^* . All other delivery batches are identical and delivered at the same time in σ as in σ^* . Only job j is delivered later in σ than in σ^* . However, since jobs j and i are delivered at the same time in σ and $d_j > d_i$ we have $L_j(\sigma) < L_i(\sigma) = L_i(\sigma^*)$, which establishes that σ is another optimal schedule. A finite number of repetitions of this argument establishes that there exists an optimal schedule in which the jobs are sequenced in EDD order. $\square$

As a result of Lemma 5, we assume throughout this subsection that the jobs are indexed in EDD order, i.e. such that $d_1 \leq \dots \leq d_n$. Hall and Potts (2003) provide an $O(n^3)$ time algorithm for problem $1||Dy + L_{\max}$. For the constrained version of this problem, $1|T|Dy + L_{\max}$, we need additional state variables.

Algorithm LC

Value Function

$f(j, h, v, y)$ = the minimum maximum lateness for processing and delivering jobs $1, \dots, j$ using y deliveries, where the last delivery when a job completes processing occurs at time P_h and there are v subsequent deliveries, so that the batch containing job j is delivered at time $P_h + vT$, where $v \leq j - h$, $h \leq j$ and $h = j$ if $v = 0$.

Boundary Conditions

$$\begin{aligned} f(0, 0, 0, 0) &= -\infty. \\ f(j, j, 0, 1) &= P_j - d_1, \quad \text{for } j > 0 \text{ and } P_j < T. \end{aligned}$$

Optimal Solution Value

$$\min_{1 \leq h \leq n, 0 \leq v \leq y \leq n} \{Dy + f(n, h, v, y)\}.$$

Recurrence Relations

$$\begin{aligned} f(j, j, 0, y) &= \min_{0 \leq h \leq i < j, 1 \leq v \leq \min\{i-h+1, V\}} \{\max\{P_j - d_{i+1}, f(i, h, v-1, y-1)\}\}, \\ &\quad \text{for } P_j \geq T. \\ f(j, h, v, y) &= \min_{0 \leq i < j} \{\max\{P_h + vT - d_{i+1}, f(i, h, v-1, y-1)\}\}, \\ &\quad \text{for } h < j \text{ and } v > V, \quad \text{where } V = \lfloor (P_j - P_h)/T \rfloor. \end{aligned}$$

The interpretation of the recurrence relations is similar to that for Algorithm CC. In both cases, the maximum lateness of any job in a newly delivered batch $\{i+1, \dots, j\}$ occurs for the job in that batch with smallest due date, i.e. job $i+1$.

Theorem 3. Algorithm LC finds an optimal schedule for problem $1|T|Dy + L_{\max}$ in $O(n^5)$ time.

Proof. The proof of optimality follows from Lemmas 1, 2 and 5. By definition, $j, h, v, y \leq n$. Consequently, the first recurrence relation with $h = j$ and $v = 0$ requires $O(n^3)$ time for each j and y . The second recurrence relation requires $O(n)$ time for each j, h, v and y . Therefore, the overall time complexity of Algorithm LC is $O(n^5)$. $\square$

2.3. Number of late jobs

We define a job to be on time if it is completed before or at its due date: otherwise, it is late. We state without proof the following adaptation of Lemma 5 to (weighted) number of late jobs problems.

Lemma 6. For problem $1|T|Dy + \sum(w_j)U_j$, there exists an optimal schedule in which the on-time jobs are sequenced according to an EDD rule.

As a result of Lemma 6, we assume throughout this subsection that the jobs are indexed in EDD order. We also make the standard assumption that $p_j \leq d_j$, for $j = 1, \dots, n$. A further assumption is that any late jobs are delivered in a separate batch at the end of the schedule. Our algorithms do not explicitly take into account the delivery of such a batch.

Hall and Potts (2003) provide an $O(n^4)$ time algorithm for problem $1||Dy + \sum U_j$. For the constrained version of this problem, $1|T|Dy + \sum U_j$, we also provide a polynomial time algorithm. Since this algorithm is difficult to describe using an explicit recurrence relation, we use the trial states description mentioned at the start of this section. We use state variables j, u and y to indicate that on-time jobs from $\{1, \dots, j\}$ have been scheduled and delivered using y deliveries, where u is the number of late jobs. Moreover, the value function $f(j, u, y)$ is the completion time of the last on-time job, and unless this is to be the final on-time job in the complete schedule only states for which the last delivery occurs at time $f(j, u, y)$ are considered explicitly.

The state (j, u, y) is extended to a trial state (j', u', y', k) , for all $j < j' \leq n$, $u \leq u' \leq u + j' - j$, $y < y' \leq y + j' - j$ and $0 \leq k < n$, by considering jobs $j + 1, \dots, j'$ of which $u' - u$ are to be late, and assuming that deliveries occur at times $f(j, u, y) + T, f(j, u, y) + 2T, \dots, f(j, u, y) + (y' - y - 1)T$ and at time t such that $d_k < t \leq d_{k+1}$ and $t \geq f(j, u, y) + (y' - y)T$. By thus fixing delivery times in advance, we know which jobs are candidates to fill the corresponding batches. If not all the candidate jobs can be processed by their delivery times, then those with largest processing times are declared late and discarded.

In the algorithm stated below, we assume that $d_0 = 0$.

Algorithm UC

Value Function

$f(j, u, y)$ = the minimum makespan for processing and delivering the on-time jobs from $\{1, \dots, j\}$, using y deliveries, given that the total number of late jobs is u , where the last delivery occurs when the final on-time job of the partial schedule completes processing except possibly for $j = n$, where $0 \leq y \leq j - u$.

Initialization

Set $f(j, j, 0) = 0$, for $j = 0, 1, \dots, n$.

Declare $(j, j, 0)$ to be an unexplored state, for $j = 0, 1, \dots, n$.

Set $f(j, u, y) = \infty$ for all other states.

Trial State Generation

If there are no unexplored states (j, u, y) with $j \leq n - 1$, then go to Optimal Solution Value.

Otherwise, choose an unexplored state (j, u, y) , where j is as small as possible, and explore it as follows:

For each $j' \in \{j + 1, \dots, n\}$, $u' \in \{u, \dots, u + j' - j\}$, $y' \in \{y + 1, \dots, y + j' - j\}$ and $k \in \{0, \dots, n - 1\}$ such that $y' = 1$ if $y = 0$, and $d_k < f(j, u, y) + (y' - y)T \leq d_{k+1}$ if $y > 0$:

Construct a trial state with label (j', u', y', k) as follows:

Set $t_h = f(j, u, y) + hT$ for $h = 1, \dots, y' - y - 1$, and set $t_{y'-y} = d_{k+1}$.

Set $t = f(j, u, y)$, $S = \emptyset$ and $\bar{u} = j - u$.

Set $S_h = \{i \mid j + 1 \leq i \leq k, t_h \leq d_i < t_{h+1}\}$ for $h = 1, \dots, y' - y - 1$, and set $S_{y'-y} = \{k + 1, \dots, j'\}$.

For $h = 1, \dots, y' - y$:

Consider jobs $i \in S_h$ one by one in SPT order, as follows:

If $t + p_i \leq t_h$, then set $S = S \cup \{i\}$, $t = t + p_i$ and $\bar{u} = \bar{u} + 1$.

Otherwise, choose $p_g = \max_{l \in S} \{p_l\}$, and if $p_i < p_g$, then set $S = (S \cup \{i\}) \setminus \{g\}$ and $t = t + p_i - p_g$.

End For

If $j' - \bar{u} > u'$, then discard the infeasible trial state (j', u', y', k) .

If $j' - \bar{u} < u'$, then select $u' + \bar{u} - j'$ jobs from S with the largest processing times, and set $S = S \setminus \{i\}$ and $t = t - p_i$ for each selected job i .

If $t < f(j, u, y) + (y' - y)T$, then unless $j' = n$ discard the infeasible trial state (j', u', y', k) .

End For

Trial State Elimination

For each trial state (j', u', y', k) :

If $t < f(j', u', y')$, then set $f(j', u', y') = t$, and declare (j', u', y') to be an unexplored state.

End For

Go to Trial State Generation.

Optimal Solution Value

Compute $\min \{u + Dy \mid 0 \leq u \leq n, 0 \leq y \leq n, f(n, u, y) < \infty\}$.

In the Trial State Generation step, the set S_h , for $h = 1, \dots, y' - y - 1$, represents those jobs that are on time if contained in the batch delivered at time t_h , but would be late in any subsequent batch. A procedure resembling the algorithm of Moore (1968) ensures that due dates are achieved for the on-time jobs. A trial state is infeasible and discarded if it contains too many late jobs, or if the last on-time job completes processing before the transporter is available. However, this final condition is not applied when $j' = n$, because the definition of the value function allows the final batch of on-time jobs to wait until the transporter becomes available before being delivered.

Theorem 4. Algorithm UC finds an optimal schedule for problem $1|T|Dy + \sum U_j$ in $O(n^8 \log n)$ time.

Proof. The proof of optimality follows from Lemmas 1, 2 and 6. Since $j, u, y \leq n$, there are $O(n^3)$ states to be explored. Each state requires trial states to be generated for each j', u', y' and k , which yields $O(n^4)$ such trial states. Therefore, the innermost

loop is executed $O(n^7)$ times. Each time, we need a heap (Kruse, 1987) to handle the discarding of the largest jobs in S , which requires $O(n \log n)$ time. Therefore, the overall time complexity of Algorithm UC is $O(n^8 \log n)$. $\square$

It follows from Karp (1972) that problem $1|T|Dy + \sum w_j U_j$ is binary NP -hard. Thus, the most efficient algorithm possible (unless $P = NP$) runs in pseudopolynomial time. Therefore, we propose a dynamic programming algorithm with state variables for the makespan and the time when the transporter becomes available, while the total cost is the function value. A partial schedule has some on-time jobs from $\{1, \dots, i-1\}$ that are processed and delivered and some on-time jobs from $\{i, \dots, j\}$ that are processed but currently undelivered.

Algorithm WC

Value Function

$f(j, i, s, t)$ = the minimum total weight of late jobs from $\{1, \dots, j\}$ plus the total delivery cost for all on-time jobs from $\{1, \dots, i-1\}$, for partial schedules in which on-time jobs from $\{1, \dots, i-1\}$ are processed and delivered, on-time jobs from $\{1, \dots, j\}$ are processed, where job i is on time if $i \leq j$, s is the time when the transporter next becomes available, and t is the makespan of the partial schedule of on-time jobs, and where $1 \leq i \leq j+1$.

Boundary Condition

$$f(0, 1, 0, 0) = 0.$$

Optimal Solution Value

$$\min_{0 \leq s < t+2T \leq P+2T} \{f(n, n+1, s, t)\}.$$

Recurrence Relations

$$\begin{aligned} f(j, i, s, t) &= \min \begin{cases} f(j-1, i, s, t-p_j), & \text{for } 1 \leq i \leq j. \\ w_j + f(j-1, i, s, t) \end{cases} & \text{if } i < j, \\ f(j, j+1, s, t) &= w_j + f(j-1, j, s, t), & \text{for } s-T < t, \\ f(j, j+1, s, t) &= \min \begin{cases} \min_{i \in I, 0 \leq s' \leq t} \{D + f(j-1, i, s', t-p_j)\}, \\ w_j + f(j-1, j, s, t), \end{cases} & \text{for } s-T = t, \\ f(j, j+1, s, t) &= \min \begin{cases} \min_{i \in I} \{D + f(j-1, i, s-T, t-p_j)\}, \\ w_j + f(j-1, j, s, t), \end{cases} & \text{for } s-T > t, \end{aligned}$$

where $I = \{i | 1 \leq i \leq j, s-T \leq d_i\}$.

The first recurrence relation considers scheduling job j to be on time or late, with no batch delivery scheduled. The second recurrence relation schedules job j to be late, since the state variables s and t prevent j being on time and a batch delivery for a batch containing job j being scheduled. The third and fourth recurrence relations schedule a delivery for the on-time jobs from $\{i, \dots, j\}$ if job j is on time. The third recurrence relation applies when the transporter is available at or before time t , so that any delivery is scheduled at time t . On the other hand, the fourth recurrence relation applies when the transporter is available after time t , so that any delivery is scheduled at time $s - T$ when the transporter is available. In each case, job j is on time in the first term and late in the second term. Since job i has the smallest due date among on-time jobs from $\{i, \dots, j\}$, the condition $s - T < d_i$ in I ensures that no job in the delivery batch is delivered after its due date.

Theorem 5. An optimal schedule for problem $1|T|Dy + \sum w_j U_j$ can be found in $O(n^2 P \min\{P, nW\})$ time.

Proof. The proof of optimality of Algorithm WC follows from Lemmas 1, 2 and 6. By definition, $j, i \leq n$ and $t \leq P$, and Lemma 3 shows that $s < P + 2T \leq 3P$. Therefore, the first recurrence relation requires $O(n^2 P^2)$ time, and the second requires $O(nP^2)$ time. The third recurrence relation, which is evaluated for all $O(nP)$ values of j and s since the value of t is defined by s , requires $O(nP)$ time. The fourth recurrence relation, which is evaluated for all $O(nP^2)$ values of j, s and t , requires $O(n)$ time. Therefore, the overall time complexity of Algorithm WC is $O(n^2 P^2)$. The existence of an alternative version of this algorithm, which uses the makespan of the partial schedule of on-time jobs as the value function, and the weighted number of late jobs and the number of deliveries as state variables, and has time complexity $O(n^3 PW)$, provides the desired result. $\square$

2.4. Total tardiness

We now consider problem $1||Dy + \sum T_j$. It follows from Du and Leung (1990) that the recognition version of this problem is binary NP -complete. We provide an optimal algorithm for this problem. This algorithm generalizes that of Lawler (1977) for the classical scheduling problem $1||\sum T_j$ and assumes that all processing times are distinct (which can be achieved by perturbation if necessary). From Lemma 2, our algorithm considers only job completion times as potential delivery times.

As in the previous two subsections, we assume that jobs are indexed in EDD order.

Algorithm TU

Value Function

$f(J(j, l, k), t, s_1, s_2)$ = the minimum total cost of processing and delivering jobs of the set $J(j, l, k)$ which comprises the jobs of $j \dots, l$ excluding those with processing time

larger than job p_k , where processing starts at time t , s_1 is the time of the first delivery at or after time t , and s_2 is the time when the last job in the set $J(j, l, k)$ is delivered, where $0 \leq t \leq s_1 \leq s_2 \leq P$ and $t + \sum_{i \in J(j, l, k)} p_i \leq s_2$.

Boundary Conditions

$f(\emptyset, t, s_1, s_2) = 0$, for $0 \leq t \leq s_1 < s_2 \leq P$.
 $f(\{j\}, t, s_1, s_2) = D + \max\{0, s_2 - d_j\}$, for $j = 1, \dots, n$ and $t \leq s_1 < t + p_j \leq s_2 \leq P$.
 $f(J(j, l, k), t, s_2, s_2) = D + \sum_{i \in J(j, l, k)} \max\{0, s_2 - d_i\}$, for $t + \sum_{i \in J(j, l, k)} p_i \leq s_2 \leq P$ and all $J(j, l, k)$.

Optimal Solution Value

$$\min_{0 \leq s_1 \leq P} \{f(\{1, \dots, n\}, 0, s_1, P)\}.$$

Recurrence Relations

$$f(J(j, l, k), t, s_1, s_2) = \min_{0 \leq e \leq |J(k', l, k)|} \{q_e\}, \quad \text{where}$$

$$q_e = \min \begin{cases} f(J_1, t, s_1, s_1) + \max\{0, s_1 - d_{k'}\} + f(J_2, C_{k'}(e), s_1, s_2), & \text{if } s_1 \geq C_{k'}(e) \\ \min_{C_{k'}(e) \leq s'_1 \leq s_2} \{f(J_1, t, s_1, s_1) + \max\{0, s'_1 - d_{k'}\} + f(J_2, C_{k'}(e), s'_1, s_2)\}, & \text{if } s_1 = C_{k'}(e) - p_{k'} \\ \min_{C_{k'}(e) \leq s'_2 \leq s_2} \{f(J_1, t, s_1, s'_2) + \max\{0, s'_2 - d_{k'}\} + f(J_2, C_{k'}(e), s'_2, s_2)\}, & \text{if } s_1 < C_{k'}(e) - p_{k'} \\ \min_{C_{k'}(e) \leq s'_1 \leq s_2} \{f(J_1, t, s_1, C_{k'}(e) - p_{k'}) + \max\{0, s'_1 - d_{k'}\} + f(J_2, C_{k'}(e), s'_1, s_2)\}, & \text{if } s_1 < C_{k'}(e) - p_{k'}, \end{cases}$$

where $C_{k'}(e) = t + \sum_{i \in J(i, k' + e, k)} p_i$, $J_1 = J(j, k' + e, k')$, $J_2 = J(k' + e + 1, l, k')$ and $k' = \operatorname{argmax}\{p_i \mid i \in J(j, l, k)\}$.

The recurrence relation constructs a decomposition of the job set $J(j, l, k)$ into three parts: a first subset, a single job k' and a second subset, as in the algorithm of Lawler (1977) for the classical version of this problem, $1 \parallel \sum T_j$. The four terms in the recurrence relation represent four possible cases. In the first and second terms, all jobs in the first subset are delivered at time s_1 , which is the first available delivery time for any of the jobs in $J(j, k, l)$. The first term considers the case where job k' is also delivered in the same batch at time s_1 , whereas the second term is for the case where s_1 is the time when all jobs in the first subset complete processing and the delivery of job k' occurs at some later time s'_1 . The third term is for the case where the final batch delivery containing jobs from the first subset also includes job k' and possibly some jobs of the second subset. Finally, the fourth term considers the case where the batch containing the last jobs of the first subset is delivered when all of these jobs complete processing, while job k' and possibly some of the jobs of the second subset are delivered at a later time s'_1 .

Theorem 6. Algorithm TU finds an optimal schedule for problem $1\|Dy + \sum T_j$ in $O(n^4 P^4)$ time.

Proof. The proof of optimality follows from Lemma 2 and analysis of the type developed by Lawler (1977). Observe that the number of possible sets $J(j, l, k)$ is $O(n^3)$ and $t, s_1, s_2 \leq P$. The optimization over e requires $O(n)$ time for each state. The optimization over s'_1 in the second and fourth terms and over s'_2 in the third term requires $O(P)$ time. Therefore, the overall time complexity of Algorithm TU is $O(n^4 P^4)$. $\square$

Problem $1|T|Dy + \sum T_j$ remains open as to pseudopolynomial time solvability.

3. Parallel machine problems

We now turn our attention to identical parallel machine scheduling environments. The scheme we use to name our algorithms is similar to that in Section 2, except that a “ P ” is inserted at the start to denote identical parallel machines. Although we consider two identical parallel machines, all of our algorithms generalize in a natural way to any constant number of identical parallel machines.

3.1. Sum of completion times

We begin by considering problems $P2\|Dy + \sum C_j$ and $P2|T|Dy + \sum C_j$.

Theorem 7. The recognition versions of problems $P2\|Dy + \sum C_j$ and $P2|T|Dy + \sum C_j$ are binary NP -complete.

Proof. Since the second problem is a generalization of the first, we consider problem $P2\|Dy + \sum C_j$. We use a reduction from the following problem which is known to be binary NP -complete.

Partition (Garey and Johnson, 1979): given m elements with integer sizes $a_1, \dots, a_m$, does there exist a partition S_1, S_2 of the index set $\{1, \dots, m\}$ such that $\sum_{i \in S_1} a_i = \sum_{i \in S_2} a_i$?

Consider an instance of problem $P2\|Dy + \sum C_j$ defined by:

$$\begin{aligned} n &= m \\ p_i &= a_i, \quad \text{for } i = 1, \dots, n, \\ D &= n \sum_{i=1}^n a_i, \quad \text{and} \\ C &= 3n \sum_{i=1}^n a_i / 2, \end{aligned}$$

where C is a threshold cost. We prove that there exists a schedule for this instance of $P2\|Dy + \sum C_j$ with cost less than or equal to C if and only if there exists a solution to Partition.

($\Rightarrow$) Let S_1, S_2 denote a solution to Partition. Schedule job j on machine M_1 for $j \in S_1$, and on machine M_2 for $j \in S_2$. Make one delivery only, at time $\sum_{i \in S_1} p_i = \sum_{i \in S_2} p_i = \sum_{i=1}^n a_i/2$ when all jobs have completed processing and can be delivered. The cost of this solution is $D + n \sum_{i=1}^n a_i/2 = C$.

($\Leftarrow$) If there does not exist a solution to Partition, then at least one job must be delivered at time $\sum_{i=1}^n a_i/2 + 1$ or later. Thus, the cost of any schedule that uses exactly one delivery must be at least $D + n(\sum_{i=1}^n a_i/2 + 1) > C$. Also, the cost of any schedule that uses two or more deliveries must be at least $2D > C$. $\square$

Problems $P2\|Dy + \sum C_j$ and $P2|T|Dy + \sum C_j$ remain open as to pseudopolynomial time solvability.

3.2. Maximum lateness

The binary NP -completeness of the recognition versions of problems $P2\|Dy + L_{\max}$ and $P2|T|Dy + L_{\max}$ follows from Lenstra, Rinnooy Kan, and Brucker (1977). We now derive a pseudopolynomial time dynamic programming algorithm for problems $P2\|Dy + L_{\max}$ and $P2|T|Dy + L_{\max}$. Lemma 5 can easily be generalized to show that the jobs on each machine are sequenced in EDD order. Thus, we assume that the jobs are indexed in EDD order.

Lemma 7. For problems $P2\|Dy + L_{\max}$ and $P2|T|Dy + L_{\max}$, there exists an optimal schedule in which the batches for delivery are formed from a partition of the sequence $(1, \dots, n)$ into subsets of jobs, and they are delivered in order of the indices of the jobs they contain.

Proof. Suppose that the batch deliveries in an optimal schedule σ^* are not of the specified form. Let j be the highest indexed job that appears in some batch B that is delivered before another batch B' containing some job i , where $j > i$. We create an alternative schedule σ in which the batch deliveries are changed from those in σ^* by removing job j from batch B and including it in batch B' . Since $d_j \geq d_i$ we deduce that $L_j(\sigma) \leq L_i(\sigma) = L_i(\sigma^*)$. Since all other jobs have the same lateness values in σ as in σ^* , we deduce that σ is also an optimal schedule. After a finite number of repetitions of this argument, we obtain an optimal schedule with the desired property. $\square$

In view of Lemma 7, a partial schedule has some jobs $1, \dots, i-1$ that are processed and delivered, and possibly some additional jobs $i, \dots, j$ that are processed but awaiting delivery. This motivates the following algorithm for problem $P2\|Dy + L_{\max}$.

Algorithm PLU

Value Function

$f(j, i, y, t_1)$ = the minimum maximum lateness for processing and delivering jobs $1, \dots, i-1$ using y deliveries, and processing jobs $i, \dots, j$, where the total processing time on machine M_1 is t_1 and where $1 \leq i \leq j+1$.

Boundary Condition

$$f(0, 1, 0, 0) = -\infty.$$

Optimal Solution Value

$$\min_{1 \leq y \leq n, 0 \leq t_1 \leq P} \{Dy + f(n, n+1, y, t_1)\}.$$

Recurrence Relations

$$f(j, i, y, t_1) = \min \begin{cases} f(j-1, i, y, t_1 - p_j), & \text{for } 1 \leq i \leq j. \\ f(j-1, i, y, t_1), \end{cases}$$

$$f(j, j+1, y, t_1) = \min \begin{cases} \min_{1 \leq i \leq j} \{\max\{t' - d_i, f(j-1, i, y-1, t_1 - p_j)\}\} \\ \min_{1 \leq i \leq j} \{\max\{t' - d_i, f(j-1, i, y-1, t_1)\}\}, \end{cases}$$

where $t' = \max\{t_1, \sum_{k=1}^j p_k - t_1\}$.

The first recurrence relation considers scheduling job j on machine M_1 or M_2 , but not delivering a batch containing jobs $i, \dots, j$. Similarly, the second recurrence relation considers scheduling job j on M_1 or M_2 and delivering a batch $\{i, \dots, j\}$ at time t' which is the makespan of the partial schedule. Although the recurrence relations consider the case where jobs $i, \dots, j$ are scheduled on the same machine and some job h completes processing after job j , where $h < i$, such a partial schedule contains more deliveries and has the same maximum lateness as the one in which the two batches containing job h and jobs $i, \dots, j$ are combined.

Theorem 8. Algorithm PLU finds an optimal schedule for problem $P2 \parallel Dy + L_{\max}$ in $O(n^3 P)$ time.

Proof. The proof of optimality follows from Lemmas 1, 2, 5 and 7. Since $j, i, y \leq n$ and $t_1 \leq P$, the first recurrence relation requires $O(n^3 P)$ time. The second recurrence relation is used $O(n^2 P)$ times, and each use requires $O(n)$ comparisons, assuming that the partial sums $\sum_{k=1}^j p_k$ for $j = 1, \dots, n$ are computed in advance. Consequently, the overall time complexity of Algorithm PLU is $O(n^3 P)$. $\square$

The constrained version of this problem, $P2|T|Dy + L_{\max}$, requires an additional state variable to store the time of the last delivery, and is solved by the following algorithm.

Algorithm PLC

Value Function

$f(j, i, y, s, t_1)$ = the minimum maximum lateness for processing and delivering jobs $1, \dots, i-1$ using y deliveries, and processing jobs $i, \dots, j$, where s is the time when the transporter next becomes available, t_1 is the total processing time on machine M_1 , and where $1 \leq i \leq j+1$.

Boundary Condition

$$f(0, 1, 0, 0, 0) = -\infty.$$

Optimal Solution Value

$$\min_{1 \leq y \leq n, P/2+T \leq s < P+2T, 0 \leq t_1 \leq P} \{Dy + f(n, n+1, y, s, t_1)\}.$$

Recurrence Relations

$$\begin{aligned} f(j, i, y, s, t_1) &= \min \begin{cases} f(j-1, i, y, s, t_1 - p_j), & \text{for } 1 \leq i \leq j. \\ f(j-1, i, y, s, t_1) \end{cases} \\ f(j, j+1, y, s, t_1) &= \min \begin{cases} \min_{1 \leq i \leq j, 0 \leq s' \leq s-T} \{\max\{t' - d_i, f(j-1, i, y-1, s', t_1 - p_j)\}\}, & \text{if } s - T = t', \\ \min_{1 \leq i \leq j, 0 \leq s' \leq s-T} \{\max\{t' - d_i, f(j-1, i, y-1, s', t_1)\}\}, & \end{cases} \\ f(j, j+1, y, s, t_1) &= \min \begin{cases} \min_{1 \leq i \leq j} \{\max\{t' - d_i, f(j-1, i, y-1, s-T, t_1 - p_j)\}\}, & \text{if } s - T > t', \\ \min_{1 \leq i \leq j} \{\max\{t' - d_i, f(j-1, i, y-1, s-T, t_1)\}\}, & \end{cases} \end{aligned}$$

where $t' = \max\{t_1, \sum_{k=1}^j p_k - t_1\}$.

The first recurrence relation is similar to that in Algorithm PLU. The second and third recurrence relations consider scheduling job j on machine M_1 or M_2 and delivering the batch comprising jobs $i, \dots, j$. The second recurrence relation applies when the transporter is available at or before time t' , where t' is the makespan of the partial schedule, so that the delivery is scheduled at time t' . On the other hand, the third recurrence relation applies when the transporter becomes available after time t' , so that batch $\{i, \dots, j\}$ is delivered at time $s - T$ when the transporter is available.

Theorem 9. Algorithm PLC finds an optimal schedule for problem $P2|T|Dy + L_{\max}$ in $O(n^3 P^2)$ time.

Proof. The proof of optimality follows from Lemmas 1, 2, 5 and 7. We observe that $j, i, y \leq n$, $t_1 \leq P$, and $s < P + 2T \leq 3P$ from Lemma 3. Thus, the first recurrence relation requires $O(n^3 P^2)$ time. The second recurrence relation is used $O(n^2 P)$ times, since the value of t_1 is defined by s , and each use requires $O(nP)$ time. The third

recurrence relation is used $O(n^2 P^2)$ times, and each use requires $O(n)$ time. Therefore, the overall time complexity of Algorithm PLC is $O(n^3 P^2)$. $\square$

3.3. Number of late jobs

The binary NP -completeness of the recognition versions of problems $P2 \parallel Dy + \sum (w_j)U_j$ and $P2|T|Dy + \sum (w_j)U_j$ follows from Lenstra, Rinnooy Kan, and Brucker (1977). We now derive a pseudopolynomial time dynamic programming algorithm for these two problems. As in Section 2.3, we assume that $p_j \leq d_j$ for $j = 1, \dots, n$, and that any late jobs are delivered in a separate batch at the end of the schedule. Our algorithms do not explicitly take into account the delivery of such a batch.

Lemma 6 can easily be generalized to show that the on-time jobs on each machine are sequenced in EDD order. Thus, we assume that the jobs are indexed in EDD order. We also state without proof a result analogous to Lemma 7.

Lemma 8. For problems $P2 \parallel Dy + \sum w_j U_j$ and $P2|T|Dy + \sum w_j U_j$, there exists an optimal schedule in which the batches of on-time jobs for delivery are formed from a partition of the sequence $(1, \dots, n)$ into subsets of jobs, where late jobs are removed, and where these batches are delivered in the order of the indices of the jobs they contain.

We now describe a dynamic programming algorithm for problem $P2 \parallel Dy + \sum w_j U_j$.

Algorithm PUU

Value Function

$f(j, i, \omega, y, t_1)$ = the minimum time at which the last on-time job on machine M_2 completes processing, for partial schedules in which on-time jobs from $\{1, \dots, i-1\}$ are processed and delivered using y deliveries and on-time jobs from $\{i, \dots, j\}$ are processed, with total weight of late jobs ω , where job i is on time if $i \leq j$, t_1 is the time when the last on-time job on machine M_1 completes processing, and where $1 \leq i \leq j+1$.

Boundary Condition

$$f(0, 1, 0, 0, 0) = 0.$$

Optimal Solution Value

$$\min \{Dy + \omega \mid 1 \leq y \leq n, 0 \leq \omega \leq W, \min_{0 \leq t_1 \leq P} \{f(n, n+1, \omega, y, t_1)\} < \infty\}.$$

Recurrence Relations

$$f(j, i, \omega, y, t_1) = \min \begin{cases} f(j-1, i, \omega, y, t_1 - p_j), \\ f(j-1, i, \omega, y, t_1) + p_j, \\ f(j-1, i, \omega - w_j, y, t_1), \end{cases} \quad \begin{matrix} \text{for } 1 \leq i \leq j. \\ \\ \text{if } i < j, \end{matrix}$$

$$f(j, j+1, \omega, y, t_1) = \min \begin{cases} \min_{i \in I_1} \{f(j-1, i, \omega, y-1, t_1 - p_j)\} \\ \min_{i \in I_2} \{f(j-1, i, \omega, y-1, t_1) + p_j\} \\ f(j-1, j, \omega - w_j, y, t_1), \end{cases}$$

where:

$$I_1 = \{i \mid 1 \leq i \leq j, \max\{t_1, f(j-1, i, \omega, y-1, t_1 - p_j)\} \leq d_i\}, \quad \text{and}$$

$$I_2 = \{i \mid 1 \leq i \leq j, \max\{t_1, f(j-1, i, \omega, y-1, t_1) + p_j\} \leq d_i\}.$$

The first recurrence relation considers scheduling job j to be on time on machine M_1 or M_2 or to be late, with no batch delivery scheduled. The second recurrence relation considers scheduling job j to be on time on machine M_1 or M_2 and delivering a batch containing on-time jobs from $\{i, \dots, j\}$ at the makespan of the partial schedule, or scheduling job j to be late. When a batch delivery is scheduled, the conditions specified in I_1 and I_2 ensure that no job from the batch is delivered after its due date.

Theorem 10. An optimal schedule for problem $P2 \parallel Dy + \sum w_j U_j$ can be found in $O(n^2 P \min\{P, nW\})$ time.

Proof. The proof of optimality of Algorithm PUU follows from Lemmas 1, 2, 6 and 8. Since $j, i, y \leq n, \omega \leq W$ and $t_1 \leq P$, the first recurrence relation requires $O(n^3 PW)$ time. The second recurrence relation is used $O(n^2 PW)$ times, and each use requires $O(n)$ time. Consequently, the overall time complexity of Algorithm PUU is $O(n^3 PW)$. The existence of an alternative version of this algorithm, which has the time when the last on-time job on machine M_2 completes processing as a state variable, and the weighted number of late jobs plus the total delivery cost as the value function, shows that the problem is also solvable in $O(n^2 P^2)$ time. $\square$

We now describe an algorithm for the equivalent constrained problem, $P2|T|Dy + \sum w_j U_j$.

Algorithm PUC

Value Function

$f(j, i, \omega, y, s, t_1)$ = the minimum time at which the last on-time job on machine M_2 completes processing, for partial schedules in which on-time jobs from $\{1, \dots, i-1\}$ are processed and delivered using y deliveries and on-time jobs from $\{i, \dots, j\}$ are processed, with total weight of late jobs ω , where job i is on time if $i \leq j$, s is the time when the transporter next becomes available, t_1 is the time when the last on-time job on machine M_1 completes processing, and where $1 \leq i \leq j+1$.

Boundary Condition

$$f(0, 1, 0, 0, 0, 0) = 0.$$

Optimal Solution Value

$$\min\{Dy + \omega \mid 1 \leq y \leq n, 0 \leq \omega \leq W, \min_{0 \leq s < P+2T, 0 \leq t_1 \leq P} \{f(n, n+1, \omega, y, s, t_1)\} < \infty\}.$$

Recurrence Relations

$$\begin{aligned} & f(j, i, \omega, y, s, t_1) \\ &= \min \begin{cases} f(j-1, i, \omega, y, s, t_1 - p_j), \\ f(j-1, i, \omega, y, s, t_1) + p_j, \\ f(j-1, i, \omega - w_j, y, s, t_1), \end{cases} \quad \text{for } 1 \leq i \leq j, \\ & f(j, j+1, \omega, y, s, t_1) \\ &= \min \begin{cases} \min_{(i, s') \in S_1} \{f(j-1, i, \omega, y-1, s', t_1 - p_j)\} \\ \min_{(i, s') \in S_2} \{f(j-1, i, \omega, y-1, s', t_1) + p_j\} \\ \min_{i \in I_1} \{f(j-1, i, \omega, y-1, s-T, t_1 - p_j)\}, & \text{if } t_1 < s-T \\ \min_{i \in I_2} \{f(j-1, i, \omega, y-1, s-T, t_1) + p_j\}, & \text{if } t_1 < s-T \\ f(j-1, j, \omega - w_j, y, s, t_1), \end{cases} \end{aligned}$$

where

$$\begin{aligned} S_1 &= \{(i, s') \mid 1 \leq i \leq j, 0 \leq s' \leq \max\{t_1, f(j-1, i, \omega, y-1, s', t_1 - p_j)\} = s-T \leq d_i\}, \\ S_2 &= \{(i, s') \mid 1 \leq i \leq j, 0 \leq s' \leq \max\{t_1, f(j-1, i, \omega, y-1, s', t_1) + p_j\} = s-T \leq d_i\}, \\ I_1 &= \{i \mid 1 \leq i \leq j, \max\{t_1, f(j-1, i, \omega, y-1, s-T, t_1 - p_j)\} < s-T \leq d_i\}, \text{ and} \\ I_2 &= \{i \mid 1 \leq i \leq j, \max\{t_1, f(j-1, i, \omega, y-1, s-T, t_1) + p_j\} < s-T \leq d_i\}. \end{aligned}$$

The first recurrence relation is similar to that in Algorithm PUU. The second recurrence relation applies when there are no jobs that are processed but undelivered. The first four terms consider scheduling job j on machine M_1 or M_2 and delivering a batch comprising on-time jobs from $\{i, \dots, j\}$. The first and second terms apply where the transporter is available when all jobs of the partial schedule have completed processing. The conditions specified in S_1 and S_2 guarantee such transporter availability and ensure that the batch delivery is scheduled so that all jobs in the batch are delivered by their due dates. The third and fourth terms are similar except that they apply when the transporter is unavailable at the time when all jobs in the partial schedule complete processing. Finally, the fifth term schedules job j to be late, in which case no jobs among $\{1, \dots, j-1\}$ are processed and undelivered.

Theorem 11. An optimal schedule for problem $1|T|Dy + \sum w_j U_j$ can be found in $O(n^2 P^3 \min\{P, nW\})$ time.

Proof. The proof of optimality of Algorithm PUC follows from Lemmas 1, 2, 6 and 8. We observe that $j, i, y \leq n$, $\omega \leq W$ and $t_1 \leq P$, and $s < P + 2T \leq 3P$ from Lemma 3. Thus, the first recurrence relation requires $O(n^3 P^2 W)$ time. The second recurrence relation is used $O(n^2 P^2 W)$ times, and each use requires $O(nP)$ time. Therefore, the overall time complexity of Algorithm PUC is $O(n^3 P^3 W)$. The existence of an alternative version of this algorithm, which has the time when the last on-time job on machine M_2 completes processing as a state variable, and the weighted number of late jobs plus the total delivery cost as the value function, shows that the problem is also solvable in $O(n^2 P^4)$ time. $\square$

4. Concluding remarks

In this paper, we present several models for scheduling problems which include a delivery cost. The inclusion of delivery cost motivates the batching of items for delivery to customers. For many of the problems considered, we provide an algorithm that runs in polynomial time. In other cases, we provide both an algorithm that runs in pseudopolynomial time and a negative result which shows that this type of algorithm is likely to be the most efficient possible. Our results demonstrate the broad applicability of dynamic programming to the solution of scheduling and batching problems. Our work has practical implications for the way in which scheduling and batch delivery decisions are made.

A number of important issues remain open for future investigation. The first is the resolution of the complexity of the few open problems described in our work. Second, while not the focus of the present work, it should be possible to design fast heuristics for the problems which we have shown to be intractable. Third, the incorporation of delivery cost into decision making in manufacturing supply chains, as discussed by Hall and Potts (2003), illustrates that delivery costs can and should be included in a wider variety of scheduling models. In conclusion, we hope that our work will lead to a greater understanding of the practical and theoretical tradeoffs involved in scheduling and batch delivery decisions.

Acknowledgments

This work is supported in part by The National Science Foundation Grant DMI-9821033, by the Summer Fellowship Program of the Fisher College of Business (The Ohio State University), by NATO Collaborative Research Grant CRG 950773, by the CORMSIS Visitor Programme (University of Southampton, U.K.), by EPSRC Grant GR/R97696/01 and by INTAS Grant 03-51-5501.

References

- Albers, S. and P. Brucker. (1993). "The Complexity of One-Machine Batching Problems." *Discrete Applied Mathematics* 47, 87–107.

- Bruno, J. and P. Downey. (1978). "Complexity of Task Sequencing with Deadlines, Set-Up Times and Changeover Costs." *SIAM Journal on Computing* 7, 393–404.
- Cheng, T.C.E., V.S. Gordon, and M.Y. Kovalyov. (1996). "Single Machine Scheduling with Batch Deliveries." *European Journal of Operational Research* 94, 277–283.
- Du, J. and J.Y.-T. Leung. (1990). "Minimizing Total Tardiness on One Machine is NP-Hard." *Mathematics of Operations Research* 15, 483–495.
- Garey, M.R. and D.S. Johnson. (1979). *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W.H. Freeman.
- Graham, R.L., E.L. Lawler, J.K. Lenstra, and A.H.G. Rinnooy Kan. (1979). "Optimization and Approximation in Deterministic Machine Scheduling: A Survey." *Annals of Discrete Mathematics* 5, 287–326.
- Hall, N.G., M.A. Lesaoana, and C.N. Potts. (2001). "Scheduling with Fixed Delivery Dates." *Operations Research* 49, 854–865.
- Hall, N.G. and C.N. Potts. (2003). "Supply Chain Scheduling: Batching and Delivery." *Operations Research* 51, 566–584.
- Ham, I., K. Hitomi, and T. Yoshida. (1985). *Group Technology: Applications to Production Management*. Boston, MA: Kluwer-Nijhoff.
- Herrmann, J. and C.-Y. Lee. (1993). "On Scheduling to Minimize Earliness-Tardiness and Batch Delivery Costs with a Common Due Date." *European Journal of Operational Research* 70, 272–288.
- Karp, R.M. (1972). "Reducibility among Combinatorial Problems." In R.E. Miller and J.W. Thatcher (eds.), *Complexity of Computer Computations*. New York: Plenum Press, pp. 85–103.
- Kruse, R.L. (1987). *Data Structures and Program Design*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall.
- Lawler, E.L. (1977). "A 'Pseudopolynomial' Time Algorithm for Sequencing Jobs to Minimize Total Tardiness." *Annals of Discrete Mathematics* 1, 331–342.
- Lee, C.-Y. and Z.-L. Chen. (2001). "Machine Scheduling with Transportation Considerations." *Journal of Scheduling* 4, 3–24.
- Lee, C.-Y., R. Uzsoy, and L.A. Martin-Vega. (1992). "Efficient Algorithms for Scheduling Semiconductor Burn-in Operations." *Operations Research* 40, 764–775.
- Lenstra, J.K., A.H.G. Rinnooy Kan, and P. Brucker. (1977). "Complexity of Machine Scheduling Problems." *Annals of Discrete Mathematics* 1, 343–362.
- Monma, C.L. and C.N. Potts. (1989). "On the Complexity of Scheduling with Batch Setup Times." *Operations Research* 37, 798–804.
- Moore, J.M. (1968). "An n Job, One Machine Sequencing Algorithm for Minimizing the Number of Late Jobs." *Management Science* 15, 102–109.
- Potts, C.N. and M.Y. Kovalyov. (2000). "Scheduling with Batching: A Review." *European Journal of Operational Research* 120, 228–249.
- Potts, C.N. and L.N. Van Wassenhove. (1992). "Integrating Scheduling with Batching and Lot-Sizing: A Review of Algorithms and Complexity." *Journal of the Operational Research Society* 43, 395–406.
- Webster, S. and K.R. Baker. (1995). "Scheduling Groups of Jobs on a Single Machine." *Operations Research* 43, 692–703.
- Yang, X. (2000). "Scheduling with Generalized Batch Delivery Dates and Earliness Penalties." *IIE Transactions* 32, 735–742.